UNIDAD DE INGENIERÍA EN COMPUTACIÓN
Sede Central Cartago
Arquitectura e Ingeniería de Datos
Profesor: Kenneth Obando

TEC | Tecnológico de Costa Rica

Proyecto Semestral · Etapa 1 (15%) Fecha de Entrega:

MediCitas

Instrucciones

Lea este documento completo antes de la primera reunión de su grupo. El proyecto se resuelve en grupos de dos o tres personas, sin excepción; los grupos de tres entregan además el módulo adicional descrito más adelante, que no otorga puntos extra. Esta etapa contempla defensa, de modo que cada integrante responde individualmente por las decisiones del grupo. Las fechas se negocian al publicarse este enunciado y luego son inamovibles; las oficiales son las publicadas en Discord.

1 Descripción

El proyecto semestral consiste en construir *MediCitas*, una plataforma *SaaS multi-tenant* de gestión de citas médicas, entendiendo por tal un producto que una empresa vende como servicio a muchas clínicas a la vez y en el que los datos de cada clínica permanecen estrictamente aislados de los demás. El trabajo se reparte en dos etapas incrementales sobre un mismo repositorio. Esta primera etapa construye el núcleo transaccional, mientras que la segunda convierte ese núcleo en una plataforma distribuida con una capa analítica, de manera que las decisiones tomadas ahora condicionan lo que después resulte barato o costoso de cambiar.

Se eligió el dominio clínico porque sus datos son información personal sensible, con lo que el eje de seguridad y privacidad del curso deja de ser una advertencia teórica y se vuelve un requisito verificable. El aislamiento entre clínicas, por su parte, obliga a resolver multi-tenancy, que es justamente donde se cruzan los esquemas de seguridad, el particionamiento y los modelos de servicio en la nube.

El alcance está acotado a propósito. No se evalúa la cantidad de funcionalidad entregada ni el acabado visual, sino que el sistema se mantenga correcto bajo acceso concurrente, que el aislamiento entre clínicas resista intentos deliberados de romperlo y que cada decisión de diseño venga acompañada de un argumento. Una decisión discutible que el grupo sostiene con evidencia reproducible recibe mejor calificación que una decisión acertada de la que nadie sabe explicar el porqué.

En las tareas cortas del curso se prohíbe importar aquello que se está aprendiendo, pero en los proyectos la regla se invierte y se trabaja con herramientas reales de la industria, ya que aquí el objetivo es la ingeniería profesional y no el estudio de los internos.

El sistema no se construye como un bloque único sobre un solo motor. La identidad se delega en un proveedor externo, de modo que el grupo no escribe su propio inicio de sesión, y la consulta de disponibilidad se apoya en una caché, porque es la que más veces se ejecuta. Trabajar con varios servicios en lugar de uno obliga a razonar sobre dónde vive cada dato y qué ocurre cuando dos copias del mismo hecho dejan de coincidir. Los almacenes documental y de grafo corresponden a la Etapa 2.

La defensa girará, tarde o temprano, alrededor de una sola pregunta, que conviene poner en el centro del diseño desde el primer día:

Dos recepcionistas reservan el último espacio de la agenda exactamente al mismo tiempo. ¿Qué hace el sistema, por qué, y cómo se demuestra?

2 El sistema

Actores

El *tenant* es la clínica. Dentro de la plataforma intervienen cuatro actores con responsabilidades distintas.

1. **Administrador de plataforma.** Pertenece a la empresa que vende el servicio, da de alta clínicas y no accede a los datos clínicos de ninguna de ellas.
2. **Administrador de clínica.** Gestiona los usuarios, los profesionales y el catálogo de servicios de su clínica, y es el único que consulta la bitácora de auditoría.
3. **Recepción.** Registra pacientes y opera la agenda, de modo que reserva, reprograma, cancela y registra el desenlace de las citas.
4. **Profesional de salud.** Consulta su propia agenda y redacta la nota clínica de las citas que atendió.

El paciente no es usuario del sistema en esta etapa. Lo atiende recepción, y esa restricción deliberada evita que el alcance crezca hacia el registro público y la confirmación por correo, que no aportan nada a lo que la etapa evalúa.

Entidades del dominio

El modelo debe sostener al menos las entidades siguientes, con los atributos que cada grupo justifique en su diccionario de datos: clínica, usuario, rol, profesional, servicio, horario de atención, bloqueo de agenda, paciente, cita, nota clínica y registro de auditoría. Las credenciales no viven en este modelo, ya que las administra el proveedor de identidad, de modo que la entidad usuario guarda únicamente el identificador que ese proveedor emite, la clínica a la que pertenece y su rol dentro de ella. Un servicio define el tipo de consulta y su duración en minutos, lo que determina el tamaño del espacio que una cita ocupa en la agenda. Un profesional atiende solo los servicios que tiene asociados.

3 Requisitos funcionales

Los requisitos que siguen definen qué hace el sistema. Están numerados para que la documentación, las pruebas y la defensa puedan referirse a ellos sin ambigüedad, y el informe de pruebas debe indicar qué prueba cubre cuál. Todos son obligatorios para grupos de dos y de tres por igual.

Administración de la plataforma y de la clínica

RF1. Dar de alta una clínica con su nombre, su identificador y su zona horaria, creando junto con ella su primer usuario administrador.

RF2. Gestionar los usuarios de una clínica, con alta, baja lógica y asignación de rol. Un usuario pertenece a exactamente una clínica, salvo el administrador de plataforma, que no pertenece a ninguna.

RF3. Mantener el catálogo de servicios de la clínica, donde cada servicio lleva nombre, duración en minutos y la especialidad que requiere.

RF4. Registrar profesionales con su especialidad y asociarles los servicios que están habilitados para atender.

Agenda y disponibilidad

RF5. Definir el horario de atención semanal de cada profesional, admitiendo varios bloques por día, por ejemplo una jornada partida en mañana y tarde.

RF6. Registrar bloqueos de agenda por vacaciones o permisos, con fecha y rango horario. Un bloqueo retira la disponibilidad hacia adelante, aunque no cancela por su cuenta las citas ya reservadas que caigan dentro de él; esas citas se listan como conflictos para que recepción las reprograme.

RF7. Calcular los espacios disponibles de un profesional en una fecha, a partir de su horario, de la duración del servicio solicitado, de sus bloqueos y de las citas vigentes. Es la consulta que más veces se ejecuta en el sistema y la que los índices deben servir.

RF8. Consultar la disponibilidad por servicio en un rango de fechas, devolviendo los profesionales que tienen espacio y cuáles son.

Ciclo de vida de la cita

RF9. Reservar una cita para un paciente, un profesional, un servicio y un instante de inicio. Esta es la operación crítica del sistema y la que la pregunta rectora interroga.

RF10. Reprogramar una cita vigente hacia otro espacio, liberando el anterior dentro de la misma transacción, de modo que en ningún momento intermedio el sistema quede con la cita duplicada ni con los dos espacios ocupados.

RF11. Cancelar una cita, dejando registro del motivo y del usuario que la canceló.

RF12. Registrar el desenlace de una cita como atendida o como ausencia, lo que solo se admite una vez pasada su hora de inicio.

RF13. Sostener los estados de la cita y rechazar cualquier transición que no esté permitida. Desde *reservada* se puede pasar a *atendida*, *ausente*, *cancelada* o *reprogramada*, y los cuatro estados finales no admiten salida.

Pacientes y ficha clínica

RF14. Registrar y actualizar los pacientes de la clínica, con identificación, nombre, fecha de nacimiento y datos de contacto.

RF15. Adjuntar una nota clínica a una cita atendida. Solo la escribe el profesional que atendió esa cita, y constituye el dato más sensible del sistema.

RF16. Consultar el historial de citas de un paciente dentro de su clínica, con el estado de cada una.

Seguridad y auditoría

RF17. Autenticar a los usuarios contra un **proveedor de identidad externo** y autorizar cada operación según el rol que viaja en el token, conforme a la matriz de permisos de la página siguiente. El servicio valida la firma del token contra las llaves públicas del proveedor y traduce sus roles a los permisos del dominio. Una solicitud sin token válido recibe 401, mientras que una con token válido pero sin el rol requerido recibe 403.

RF18. Registrar en una bitácora *append-only* toda lectura y toda escritura de notas clínicas y de datos de contacto de pacientes, anotando quién, qué, sobre cuál paciente y cuándo.

RF19. Permitir la consulta de la bitácora al administrador de la clínica, con filtros por usuario, por paciente y por rango de fechas.

Operación diaria

RF20. Mostrar la agenda del día, tanto la de un profesional como la de toda la clínica, que es la vista con la que trabaja recepción.

RF21. Contar las citas por estado dentro de un rango de fechas. Basta una consulta agregada sobre el motor transaccional, ya que la analítica real corresponde a la Etapa 2.

Matriz de permisos

La matriz fija el mínimo exigido. Cada grupo puede ampliarla si lo documenta y lo justifica, aunque no puede reducirla.

Operación	Adm. plataforma	Adm. clínica	Recepción	Profesional
Alta de clínicas (RF1)	sí	no	no	no
Usuarios, servicios y profesionales (RF2 a RF4)	no	sí	no	no
Horarios y bloqueos (RF5, RF6)	no	sí	no	solo los propios
Consultar disponibilidad (RF7, RF8)	no	sí	sí	sí
Reservar, reprogramar y cancelar (RF9 a RF11)	no	sí	sí	no
Registrar desenlace (RF12)	no	sí	sí	sí
Pacientes (RF14)	no	sí	sí	solo lectura
Nota clínica (RF15)	no	no	no	solo de citas propias
Historial del paciente (RF16)	no	sí	solo lectura	sí
Bitácora de auditoría (RF19)	no	sí	no	no

4 Invariantes de negocio

Las condiciones siguientes deben ser verdaderas en todo momento, incluso cuando varias operaciones ocurren a la vez. Son el objeto de las pruebas de aislamiento y de concurrencia, y la defensa preguntará cómo se garantiza cada una.

I1. Un profesional nunca tiene dos citas vigentes cuyos intervalos se solapen.

I2. Un paciente nunca tiene dos citas vigentes solapadas dentro de la misma clínica.

I3. Toda cita vigente cae dentro del horario de atención del profesional y fuera de sus bloqueos.

I4. La duración de la cita la determina el servicio y ninguna operación la altera por su cuenta.

I5. Ningún dato de una clínica resulta accesible desde otra, por ninguna ruta ni combinación de parámetros.

I6. La bitácora de auditoría no admite modificación ni borrado por parte de la aplicación.

I7. Cancelar o reprogramar una cita libera su espacio de forma inmediata y visible para la siguiente consulta de disponibilidad.

I8. La disponibilidad que el sistema responde nunca contradice el estado de la base, aunque se sirva desde la caché. Una reserva concedida invalida de inmediato cualquier copia previa de esa agenda.

5 Núcleo obligatorio

Los requisitos funcionales definen qué debe hacer el sistema. Los seis módulos siguientes fijan con qué calidad de ingeniería ha de construirse, de manera que un sistema que cubre la funcionalidad por medios frágiles no satisface esta sección. En cada módulo se evalúan por igual el resultado técnico y el argumento que lo acompaña, entendido como qué alternativas se consideraron, qué se ganó y qué se sacrificó.

1. **Modelo de datos.** Esquema relacional normalizado hasta la tercera forma normal, con la justificación de cada paso, diagrama entidad-relación, diccionario de datos y datos semilla realistas, que deben incluir al menos tres clínicas, varios profesionales por clínica y citas repartidas en el tiempo. Toda desnormalización deliberada se documenta con su motivo.

2. **Multi-tenancy.** El aislamiento entre clínicas es el corazón del producto. El grupo compara por escrito al menos dos estrategias (esquema por *tenant*, columna discriminadora, *Row-Level Security*), elige una con argumentos y demuestra el invariante I5 con pruebas automatizadas que incluyan intentos deliberados de fuga, tales como manipular el identificador de clínica en los parámetros de consulta.

3. **Transacciones y concurrencia.** La reserva (RF9) y la reprogramación (RF10) se resuelven de forma transaccional. El grupo elige y defiende el nivel de aislamiento, maneja el conflicto y el *deadlock*, y entrega una **prueba automatizada de carrera** en la que dos clientes concurrentes compiten por el último espacio y producen exactamente una reserva concedida, sin duplicados y sin violación de I1, de forma reproducible.

4. **Identidad delegada.** La autenticación no se implementa a mano, sino que se delega en un proveedor de identidad externo, **Keycloak**, que corre como un contenedor más del Compose. El grupo configura en él un *realm* y un *client* para la aplicación, define los roles del sistema y protege las rutas exigiendo un token de acceso OIDC cuya firma el servicio valida contra las llaves públicas del proveedor. Queda por resolver, y debe quedar documentado, cómo se asocia cada identidad del proveedor con su clínica, ya que de ahí depende el invariante I5.

5. **Protección de datos y auditoría.** Roles de base de datos con menor privilegio, al menos tres perfiles distintos alineados con la matriz de permisos, consultas parametrizadas en la totalidad del código y la bitácora *append-only* de RF18 protegida a nivel de motor, no solo por convención de la aplicación. El cifrado en tránsito y el endurecimiento del despliegue corresponden a la Etapa 2.

6. **Caché de disponibilidad.** La consulta de RF7 se sirve desde **Redis**, que también corre en el Compose. El grupo decide la clave, el alcance de cada entrada y la política de invalidación, y sostiene el invariante I8 con una prueba automatizada que reserve y vuelva a consultar. El informe documenta qué se midió antes y después de introducir la caché, aunque basta una comparación sencilla, ya que las pruebas de carga corresponden a la Etapa 2. Usar Redis como almacén primario de algo no está permitido, porque la fuente de verdad sigue siendo el motor relacional.

7. **API del dominio.** Un servicio con API REST que exponga los requisitos funcionales anteriores. No se exige interfaz gráfica, de modo que la demostración se hace contra la API con un cliente de línea de comandos o con una colección de peticiones versionada en el repositorio.

8. **Pruebas e índices.** Pruebas unitarias que aíslan la lógica y pruebas de integración que ejercitan la API contra la pila real que levanta Compose, incluidas las de aislamiento, la de carrera y al menos una por cada invariante. Los índices creados se justifican con el plan de ejecución de la consulta que pretenden servir, empezando por RF7. Las pruebas de carga corresponden a la Etapa 2.

6 Requisitos de operación

El lenguaje y el marco de trabajo son libres, pero el modo de ejecución no, ya que de él dependen la comparabilidad entre grupos y la posibilidad de evaluar sin intervención del equipo.

1. **Arranque con un solo comando.** Tras copiar `.env.example` a `.env`, un único `docker compose up` levanta los cuatro servicios (la aplicación, el motor relacional, Keycloak y Redis), deja el esquema creado, los datos semilla cargados y el *realm* de Keycloak configurado con sus roles y usuarios de prueba, sin pasos manuales ni carga de archivos a mano. Los servicios se alcanzan entre sí por su nombre dentro de la red que Compose crea, nunca por una dirección fija.
2. **Contrato documentado.** Cada ruta de la API declara su método, su entrada, su salida y sus códigos de estado. La reserva distingue sus tres desenlaces: concedida, espacio ya tomado y solicitud inválida.
3. **Manejo de errores.** El servicio no se interrumpe ante una entrada mal formada ni ante un recurso inexistente, sino que responde con el código que corresponde y permanece operativo.
4. **Pruebas ejecutables con un comando documentado**, sin pasos intermedios y sin preparación manual de datos.

7 Criterios de aceptación

Siete comprobaciones deciden si la etapa está funcionando, y todas se ejecutan sobre el repositorio recién clonado, antes de la defensa.

1. **Levanta solo.** `docker compose up` deja el sistema operativo y poblado, sin intervención del grupo.
2. **El ciclo de la cita se cumple.** Reservar, reprogramar, cancelar y registrar el desenlace funcionan desde la API con los códigos de estado declarados, y una transición no permitida se rechaza.
3. **La identidad se exige y se respeta.** Una ruta protegida sin token responde `401`, con un token de un rol que no corresponde responde `403`, y con un token válido del rol adecuado funciona.
4. **El aislamiento se sostiene.** Una sesión autenticada en una clínica no obtiene datos de otra por ninguna ruta, incluidas las de listado con parámetros manipulados.
5. **La caché no miente.** Consultar la disponibilidad, reservar uno de los espacios devueltos y volver a consultar muestra el espacio ya tomado, sin esperar a que expire ninguna entrada.
6. **La carrera tiene una sola ganadora.** La prueba de concurrencia corre de forma reproducible y arroja exactamente una reserva concedida, sin violar I1.
7. **Las pruebas corren.** El comando documentado ejecuta las unitarias y las de integración, y todas pasan.

8 Herramientas y límites

En este proyecto se usan herramientas reales, entre ellas un motor relacional, Redis, Keycloak, Docker, Docker Compose, el marco web que el grupo prefiera, un ORM si lo considera conveniente, las bibliotecas de cliente de OIDC y las de prueba del lenguaje elegido. Ninguna de esas piezas hay que construirla.

En identidad la frontera se invierte respecto de las tareas cortas, porque aquí lo correcto es no escribir el mecanismo a mano. No se acepta implementar el inicio de sesión, la emisión ni la firma de tokens por cuenta propia, y el

trabajo del grupo consiste en configurar el proveedor y validar contra él lo que recibe. Con la caché ocurre algo parecido, ya que construir un almacén llave-valor es el objeto de una tarea corta y no de este proyecto; aquí se usa Redis tal cual y lo que se evalúa es la política de invalidación.

Lo que no se delega es el diseño ni su sustento. El modelo de datos, la estrategia de multi-tenancy, el control de concurrencia y la prueba que lo demuestra son trabajo del grupo, y un componente que el grupo no sepa fundamentar no se contabiliza, aunque funcione. Cuando una herramienta decide algo por omisión, como el nivel de aislamiento que el ORM configura sin que nadie se lo pida, el grupo debe saber cuál quedó activo y por qué le sirve.

9 Módulo adicional para grupos de tres

Los grupos de tres personas implementan exactamente uno de los módulos siguientes, con el mismo estándar de calidad del núcleo:

1. **Auditoría consultable y verificable.** Además de lo que pide RF19, un mecanismo que permita detectar manipulación de la bitácora, por ejemplo un encadenamiento de huellas entre registros consecutivos, con la prueba que lo demuestre.
2. **Row-Level Security como segunda estrategia.** Implementar RLS en paralelo a la estrategia de multi-tenancy elegida y comparar ambas sobre las mismas consultas, con pruebas de aislamiento y mediciones.
3. **Gestión de adjuntos clínicos.** Archivos asociados a la cita o al paciente, en BLOB o en almacenamiento externo, con control de acceso por clínica y por rol, y con su registro en la bitácora.
4. **Estrategias de invalidación comparadas.** Implementar dos políticas distintas para la caché de disponibilidad, por ejemplo escritura directa frente a invalidación por evento, y compararlas sobre el mismo escenario concurrente, reportando tanto la tasa de aciertos como los casos en que cada una llega a violar el invariante I8.

Este módulo **no otorga puntos extra**, ya que constituye el alcance esperado de un grupo de tres personas. Un grupo de tres que entregue solo el núcleo obtiene el nivel Deficiente en ese criterio, sin redistribución.

10 Principios de evaluación

Esta etapa se rige por los dos principios transversales del curso, que la rúbrica recoge con peso propio.

El **Principio de Defensa Técnica** establece que en cada rubro el estudiante demuestra, con solvencia y completitud, las razones de sus decisiones técnicas, metodológicas, arquitecturales y profesionales. No poder sustentar un rubro se penaliza aunque el entregable funcione, y cada integrante responde de forma individual. Un error que el estudiante comprende y explica recibe mejor trato que un acierto cuyo origen nadie en el grupo es capaz de reconstruir.

El **Principio de Aportación Demostrable**, con la cláusula BDFL, permite el uso de herramientas de inteligencia artificial pero no admite que sustituyan el aprendizaje. Si la aportación intelectual del grupo no resulta visible, la entrega queda sujeta a penalización, por alta que parezca su calidad. La valoración corresponde al docente, se apoya en evidencia observable, admite siempre el derecho de descargo y no recurre a detectores automáticos. Cada entrega incluye una declaración de uso de IA, y aunque declararlo no penaliza, ocultarlo agrava.

11 Fechas

La etapa arranca con la publicación de este enunciado. La fecha de entrega se negocia en clase en ese mismo momento, por lo que conviene revisar antes los cronogramas de los otros cursos, y una vez acordada queda inamovible. Las fechas oficiales son siempre las que se publican en el canal de anuncios de Discord. La entrega se realiza por el buzón de TecDigital a más tardar a las 10:00 p.m. del día acordado, y cada día de atraso descuenta cinco puntos sobre base cien.

Antes de la entrega hay un **avance obligatorio sin nota**, en una sesión virtual que se anuncia con antelación. Se revisa en vivo el modelo relacional, la estrategia de multi-tenancy elegida con su comparación escrita en borrador, que el servicio levante con Docker Compose y que el repositorio tenga *commits* de todos los integrantes. La retroalimentación de ese avance no se califica, aunque desatenderla suele costar puntos en la entrega.

La defensa se realiza en la sesión virtual posterior a la entrega, según la agenda de horarios que se publica en Discord.

12 Entregables

La entrega en TecDigital consiste en un único paquete que contiene:

1. El enlace al repositorio de GitHub, con historial de *commits* de todos los integrantes. La autoría individual se determina por los *commits* y, en los archivos comunes, se considera compartida. Un integrante sin *commits* se evalúa de forma individual según la autoría que pueda demostrarse.
2. El código fuente completo, sin dependencias instaladas, de modo que el repositorio no contenga `node_modules`, `.venv`, `vendor` ni equivalentes, junto con el `docker-compose.yml` que levanta el sistema con un solo comando, el `.env.example` correspondiente y la configuración del *realm* de Keycloak versionada, de modo que el proveedor quede listo al arrancar y no haya que configurarlo por su consola.
3. La documentación en formato **Markdown** dentro del repositorio, que comprende el `README` con las instrucciones reproducibles y el contrato de las rutas, el documento de diseño con el diagrama entidad-relación, la comparación de estrategias de multi-tenancy y la decisión de aislamiento con sus *trade-offs*, y el informe de pruebas, que indica qué prueba cubre cuál requisito funcional y cuál invariante.
4. Los scripts de datos semilla y de pruebas, ejecutables por el evaluador.
5. Un video de demostración de máximo diez minutos, **sin edición**, que muestre el arranque del sistema, el recorrido completo de una cita desde la reserva hasta su desenlace, y la prueba de carrera ejecutándose. El video tiene un fin práctico, ya que evita que la evaluación se bloquee por un problema de entorno en la máquina del evaluador.
6. La declaración de uso de inteligencia artificial.

13 Defensa

La defensa es virtual y dura veinte minutos por grupo, repartidos en ocho de demostración, siete de preguntas técnicas y cinco de preguntas sobre decisiones, en las que cada integrante responde de forma individual. El evaluador ejecuta `docker compose up` antes de la sesión; si el sistema no levanta, la defensa procede igual y cada módulo se califica en el estado en que pudo demostrarse.

Tres preguntas están garantizadas y conviene prepararlas: la pregunta rectora del proyecto, la justificación de la estrategia de multi-tenancy y la elección del nivel de aislamiento.

14 Evaluación

La evaluación distingue el *criterio técnico demostrable* (70 puntos), que recoge lo que puede verse funcionando con evidencia reproducible, la *argumentación y defensa* (15 puntos), donde se concreta el Principio de Defensa Técnica, y la *profesionalidad* de la entrega (15 puntos). Cada criterio tiene un peso, y su nota resulta de multiplicar ese peso por el nivel de logro alcanzado según la matriz.

Rúbrica de la Etapa 1

Criterio (peso)	Excelente (100 %)	Bueno (75 %)	Regular (50 %)	Deficiente ($\leq$25 %)
Modelo de datos (7)	3FN justificada paso a paso, ER y diccionario completos, semilla con varias clínicas; toda desnormalización argumentada.	Modelo correcto con justificación parcial o diccionario incompleto.	Esquema funcional con anomalías de diseño sin documentar.	Modelo que no sostiene el dominio, o ausencia de ER y diccionario.
Funcionalidad del dominio (10)	Los 21 requisitos funcionales operan desde la API con la matriz de permisos aplicada; las transiciones de estado inválidas se rechazan.	Falta algún requisito secundario o la matriz de permisos se aplica de forma incompleta.	Faltan requisitos del ciclo de la cita, o los permisos no se verifican en el servidor.	El ciclo de la cita no se completa.
Multi-tenancy (9)	Dos estrategias comparadas por escrito, una elegida con argumentos, e I5 demostrado con pruebas que incluyen intentos de fuga.	Estrategia implementada y probada, con comparación superficial.	Aislamiento implementado sin pruebas que lo verifiquen.	Los datos de una clínica son alcanzables desde otra.
Transacciones y concurrencia (12)	Prueba de carrera reproducible con una sola ganadora; nivel de aislamiento elegido, nombrado y defendido; conflicto y deadlock manejados; I1, I2 e I7 sostenidos.	Prueba presente y correcta, con la elección del nivel sin argumentar.	La operación es transaccional pero la prueba no es reproducible o no discrimina.	La carrera admite dos ganadoras, o no existe prueba.
Identidad delegada (7)	Realm y client configurados por código al levantar, firma validada contra el proveedor, roles del token traducidos a la matriz, y la clínica derivada de la identidad.	Integración correcta con alguna configuración manual o roles mapeados de forma incompleta.	Valida el token de forma superficial, o la clínica llega en la petición sin verificarse.	Autenticación escrita a mano, o rutas protegidas que aceptan cualquier token.
Protección de datos y auditoría (8)	Tres roles de base con menor privilegio alineados con la matriz, parametrización total y bitácora append-only protegida en el motor.	Controles presentes con un descuido menor.	Roles o auditoría implementados de forma incompleta.	Consultas concatenadas, o bitácora modificable desde la aplicación.
Caché de disponibilidad (6)	Clave y alcance justificados, política de invalidación explícita, I8 demostrado con prueba automatizada y medición antes y después.	Caché funcional e invalidada, con medición ausente o superficial.	Caché que depende solo de expiración por tiempo, con ventanas de incoherencia admitidas sin análisis.	La caché devuelve disponibilidad falsa, o Redis se usa como fuente de verdad.
Pruebas e índices (6)	Unitarias y de integración ejecutables con un comando, con al menos una prueba por invariante y la trazabilidad a los RF; índices justificados con el plan de ejecución.	Cobertura adecuada con índices sin justificar o sin trazabilidad.	Pruebas escasas o que no ejercitan la pila real.	Sin pruebas automatizadas.
Módulo de grupos de 3 (5)	Módulo elegido implementado con el estándar del núcleo y documentado.	Implementado con documentación incompleta.	Implementación parcial.	No se entregó.
Argumentación y defensa (15)	Decisiones sustentadas con alternativas y trade-offs; cada integrante domina de forma individual lo que presenta.	Sustento sólido con algún integrante que depende del grupo.	Justificaciones genéricas o repartidas de forma desigual.	El grupo no sostiene sus propias decisiones.
Profesionalidad (15)	Arranque reproducible con un comando, código ordenado y con estándares, repositorio con commits de todos, documentación Markdown completa, video conforme y entrega según las reglas del curso.	Entrega conforme con algún descuido de orden o documentación.	Requiere ajustes en la máquina del evaluador, contribución desigual o video parcial.	No levanta sin intervención del grupo, o entrega que incumple las reglas.

Grupos de dos. El criterio del módulo para grupos de tres no aplica y sus 5 puntos se redistribuyen en el bloque técnico, con lo que modelo de datos pasa a 8, funcionalidad del dominio a 11, multi-tenancy a 10, transacciones y concurrencia a 13, y caché de disponibilidad a 7. Un grupo de tres que no entregue ese módulo obtiene el nivel Deficiente en él.

Uso de inteligencia artificial. El uso de herramientas de IA está permitido, pero cada entrega incluye una breve declaración de cómo se usaron, en qué partes y qué se verificó de forma manual. Declararlo no penaliza, mientras que ocultarlo agrava la falta. Si en la revisión no se distingue la aportación intelectual del grupo, porque el trabajo aparente haber sido producido de forma exclusiva por una herramienta automática, la entrega queda sujeta a penalización, según el Principio de Aportación Demostrable y la cláusula BDFL de las disposiciones del curso.

Aplican además las penalizaciones transversales del curso. Se descuentan cinco puntos por cada día de atraso sobre base cien. La ausencia de *commits* de un integrante convierte su evaluación en individual, según la autoría que pueda demostrarse. Subir al repositorio las carpetas de dependencias ya instaladas descuenta cinco puntos del bloque de profesionalidad.